ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

BÁO CÁO BÀI TẬP LỚN

MÔN VI XỬ LÝ - VI ĐIỀU KHIỂN

Mã môn học: CO3009

GVHD: Lê Trọng Nhân

Sinh viên thực hiện:

Họ và Tên	**MSSV**
Nguyễn Văn Hiếu	2310967
Nguyễn Minh Hạnh	2310895
Trần Đình Hoàng	2311075

TP.HCM, tháng 12, 2025

Bảng phân công nhiệm vụ:

Họ và Tên	**MSSV**	**Nhiệm vụ**	**Hoàn thành**
Nguyễn Văn Hiếu	2310967	• Task LED, test trên mạch thật	100%
Nguyễn Minh Hạnh	2310895	• Task LCD I2C, test trên mạch thật	100%
Trần Đình Hoàng	2311075	• Task Schedule, bổ sung chế độ người đi bộ	100%

Mã nguồn (GitHub): https://github.com/VanHieu205/MCU_MPU-Assignment.git

Mục lục

1 Phân tích thiết kế **4**

1.1 Hệ thống đèn giao thông 4

1.1.1 Trạng thái bình thường (NOMAL MODE) 4

1.1.2 Trạng thái điều khiển bằng tay (MANUAL MODE) . 5

1.1.3 Trạng thái cài đặt thời gian cho 3 đèn (MODIFY MODE) 5

1.1.4 Chế độ cho người đi bộ 6

1.2 Thiết kế bộ định thời(Schedule) 7

1.2.1 Cấu trúc Schedule 7

1.2.2 Dùng sched để lên lịch cho 1 số hoạt động 7

1.3 Thiết kế LCD 11

1.3.1 Mục đích sử dụng LCD 11

1.3.2 Phần cứng và giao tiếp 11

1.3.3 Nguyên lý hiển thị 11

1.3.4 Nội dung hiển thị theo từng chế độ 12

2 Kiểm thử và mô phỏng **13**

2.1 Dùng protues mô phỏng với STM32F103C6 13

2.2 Chạy trên mạch thật 14

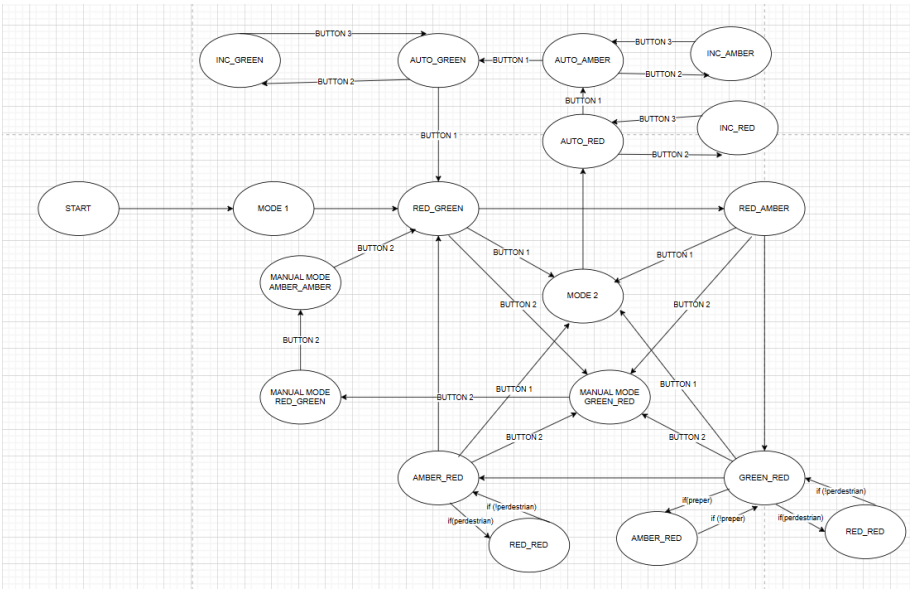
3 Kết luận **18**

LỜI CẢM ƠN

Lời đầu tiên, nhóm chúng em xin gửi lời tri ân sâu sắc đến Thầy Lê Trọng Nhân. Với sự hướng dẫn tận tình và những định hướng chuyên môn quý báu, Thầy đã giúp nhóm xác định rõ mục tiêu, tháo gỡ các vướng mắc trong quá trình nghiên cứu và hoàn thiện bài tập lớn này. Những kiến thức và kinh nghiệm mà Thầy truyền đạt không chỉ giúp nhóm hoàn thành đề tài mà còn là hành trang quý giá cho chúng em trong quá trình học tập sau này.

1 Phân tích thiết kế

1.1 Hệ thống đèn giao thông



Hình 1: Máy trạng thái cho hệ thống đèn

1.1.1 Trạng thái bình thường (NOMAL MODE)

Hệ thống khởi tạo chuyển đến MODE 1 -> trạng thái RED_GREEN, hết thời gian đèn xanh chuyển sang trạng thái RED_AMBER, hết thời gian đèn vàng chuyển trạng thái GREEN_RED, hết thời gian đèn xanh chuyển trạng thái AMBER_RED, hết thời gian đèn vàng chuyển trạng thái RED_GREEN.

- Trạng thái RED_GREEN: đèn đường 1 đỏ, đường thứ 2 xanh,
- Trạng thái RED_AMBER: đèn đường 1 đỏ, đường thứ 2 vàng
- Trạng thái GREEN_RED: đèn đường 1 xanh, đường thứ 2 đỏ
- Trạng thái RED_GREEN: đèn đường 1 vàng, đường thứ 2 đỏ

1.1.2 Trạng thái điều khiển bằng tay (MANUAL MODE)

Khi nhấn button 2 hệ thống chuyển sang trạng thái MANUAL MODE GREEN_RED, tiếp tục nhấn button 2 chuyển trạng thái MANUAL MODE RED_GREEN, tiếp tục nhấn button 2 chuyển trạng thái MANUAL MODE AMBER_AMBER, tiếp tục nhấn button về lại NOMAL MODE tại trạng thái RED_GREEN

- Trạng thái MANUAL MODE GREEN_RED: đèn đường 1 xanh, đường thứ 2 đỏ
- Trạng thái MANUAL MODE RED_GREEN: đèn đường 1 đỏ, đường thứ 2 xanh
- Trạng thái MANUAL MODE AMBER_AMBER: đèn đường 1 vàng, đường thứ 2 vàng

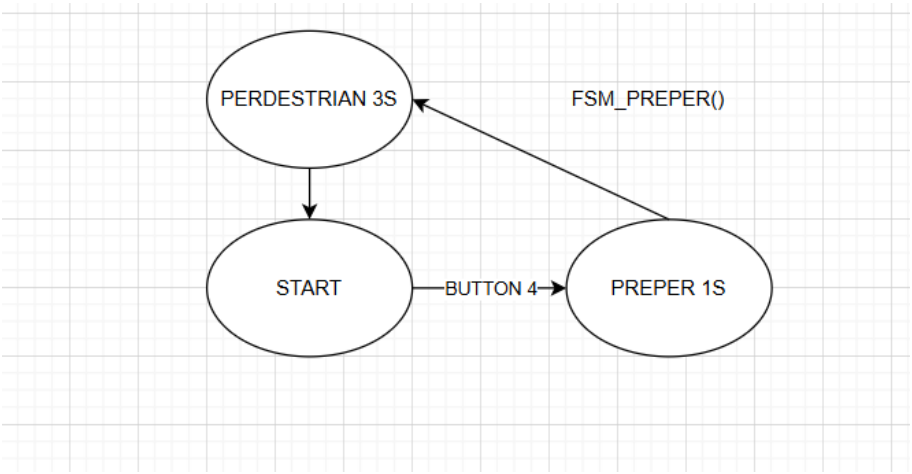
1.1.3 Trạng thái cài đặt thời gian cho 3 đèn (MODIFY MODE)

Khi nhấn button 1 hệ thống chuyển sang MODE 2 -> trạng thái AUTO_RED.
Ở trạng thái AUTO_RED, nếu nhấn button 2 sang INC_RED (ở INC_RED nếu nhấn button 3 về lại AUTO_RED), còn nhấn button 1 chuyển sang trạng thái AUTO_AMBER.
Ở trạng thái AUTO_AMBER, nếu nhấn button 2 sang INC_AMBER (ở INC_AMBER nếu nhấn button 3 về lại AUTO_AMBER), còn nhấn button 1 chuyển sang trạng thái AUTO_GREEN.
Ở trạng thái AUTO_GREEN, nếu nhấn button 2 sang INC_GREEN (ở INC_GREEN nếu nhấn button 3 về lại AUTO_GREEN), còn nhấn button 1 về lại MODE 1 trạng thái RED_GREEN.

- Trạng thái AUTO RED: đèn cả 2 tuyến sáng đỏ
- Trạng thái AUTO AMBER: đèn cả 2 tuyến sáng vàng
- Trạng thái AUTO GREEN: đèn cả 2 tuyến sáng xanh

- Trạng thái INC_RED: đèn cả 2 tuyến sáng đỏ, nếu nhấn button 2 tăng thời gian đèn đỏ, còn nhấn button 3 giảm thời gian đèn đỏ
- Trạng thái INC_AMBER: đèn cả 2 tuyến sáng vàng, nếu nhấn button 2 tăng thời gian đèn vàng, còn nhấn button 3 giảm thời gian đèn vàng
- Trạng thái INC_GREEN: đèn cả 2 tuyến sáng xanh, nếu nhấn button 2 tăng thời gian đèn xanh, còn nhấn button 3 giảm thời gian đèn xanh

1.1.4 Chế độ cho người đi bộ



Hình 2: Máy trạng thái cho hành động người đi bộ

Khởi tạo 1 máy trạng thái đọc tin hiệu button 4 để chuyển trạng thái PREPER 1S, cờ preper = true, hết 1 giây chuyển sang trạng thái PERDESTRIAN 3S, cờ perdestrian = true và tắt cở preper, hết 3 giây về trạng thái START, tắt cở perdestrian chờ tín hiệu button 4.
Khi ở NOMAL MODE trạng thái GREEN_RED kiểm nếu preper = true thì hiện trạng thái AMBER_RED, còn nếu perdestrian = true thì trạng thái RED_RED.
Khi ở NOMAL MODE trạng thái AMBER_RED kiểm nếu perdestrian = true thì trạng thái RED_RED.

- Trạng thái RED_RED: đèn đường 1 đỏ, đường thứ 2 đỏ,

- Trạng thái AMBER_RED: đèn đường 1 vàng, đường thứ 2 đỏ

1.2 Thiết kế bộ định thời(Schedule)

1.2.1 Cấu trúc Schedule

Cấu trúc bộ định thời được thiết kế theo cấu trúc linkedlist và tính toán lại mối quan hệ thời gian giữa các task để tối ưu hàm update O(1).
VD: Các task A(2) chạy sau 2s, B(3), C(5) thay vì lưu A(2)-B(3)-C(5) thì lưu thành A(2)-B(1)-C(2)

```
typedef struct Task_Control {
 void (*Task_ptr)();
 uint32_t TaskID;
 uint32_t Delay;
 uint32_t Period;
 struct Task_Control* nextTask;
}Task_Control;

typedef struct {
 Task_Control* head;

 uint32_t size;
}Task_List;
```

1.2.2 Dùng sched để lên lịch cho 1 số hoạt động

```
#define NORMAL_STATE            SET
#define PRESSED_STATE           RESET
#define NUM_OF_BUTTONS          4

int isButtonPressed(int index);
void getKeyInput();

void toggleButton0();
void toggleButton1();
void toggleButton2();
void toggleButton3();
```

Hình 3: Cấu trúc button.h khởi tạo các hành động nhấn nút

Hàm setTime: Dùng setTime vừa dùng lập lịch cài đặt, vừa kiểm tra được hoạt động của các nút nhấn mà không set qua biến toàn cục.

```
void setTime(){
         SCH_Add_Task(toggleButton0, timeset_traffic++, 0);
         if(TimeRED) {
          if(inc_red){
           for(int i = 0; i < TimeRED - RED; i++)
               SCH_Add_Task(toggleButton1, timeset_traffic++, 0);
          }else{
           for(int i = 0; i < RED - TimeRED; i++)
               SCH_Add_Task(toggleButton3, timeset_traffic++, 0);
          }
    SCH_Add_Task(toggleButton2, timeset_traffic++, 0);
         }
         SCH_Add_Task(toggleButton0, timeset_traffic++, 0);
         if(TimeAmber) {
          if(inc_amber){
           for(int i = 0; i < TimeAmber - AMBER; i++)
               SCH_Add_Task(toggleButton1, timeset_traffic++, 0);
          }else{
           for(int i = 0; i < AMBER - TimeAmber; i++)
               SCH_Add_Task(toggleButton3, timeset_traffic++, 0);
          }
          SCH_Add_Task(toggleButton2, timeset_traffic++, 0);
          }
         SCH_Add_Task(toggleButton0, timeset_traffic++, 0);
         if(TimeGreen) {
          if(inc_green){
           for(int i = 0; i < TimeGreen - GREEN; i++)
               SCH_Add_Task(toggleButton1, timeset_traffic++, 0);
          }else{
           for(int i = 0; i < GREEN - TimeGreen; i++)
               SCH_Add_Task(toggleButton3, timeset_traffic++, 0);
          }
             SCH_Add_Task(toggleButton2, timeset_traffic++, 0);
            }
         SCH_Add_Task(toggleButton0, timeset_traffic++, 0);
}
```

Hình 4: Hàm setTime để dùng đặt lịch cài đặt thời gian tự động

Cách dùng trong main: Sched có thể dùng để chạy các task về timer và button hoặc dùng nhấn button.

```
SCH_INIT();
 SCH_Add_Task(timerRun,5,1);
 SCH_Add_Task(getKeyInput,10,1);
 setTime();
 SCH_Add_Task(toggleButton1, 50,2000);
 SCH_Add_Task(toggleButton1, 51,2000);
 SCH_Add_Task(toggleButton1, 52,2000);
 SCH_Add_Task(toggleButton1, 552,2000);

 SCH_Add_Task(toggleButton3, 1000,500);
```

Hình 5: Cách dùng trong main

Trong vd này, khi hoạt động tại sau 0.5s khi bắt đầu sẽ chuyển sang MANUAL MODE AMBER_AMBER và sau 5s về lại NOMAL MODE(Dùng button 2) và chu kì lặp lại mỗi 20s. Hoạt động thứ 2 là nhấn button thứ 4 để mô phỏng người đi bộ mỗi 5s chạy lần đầu sau 10s kể từ lúc khởi động.

1.3 Thiết kế LCD

1.3.1 Mục đích sử dụng LCD

Màn hình LCD được sử dụng nhằm hiển thị trực quan trạng thái hoạt động của hệ thống đèn giao thông, giúp người dùng dễ dàng theo dõi chế độ hiện tại, trạng thái đèn của từng tuyến và thời gian cài đặt cho các màu đèn. Việc tích hợp LCD giúp tăng tính thân thiện của hệ thống và hỗ trợ quá trình kiểm thử, mô phỏng.

1.3.2 Phần cứng và giao tiếp

Hệ thống sử dụng màn hình LCD ký tự 16x2 giao tiếp với vi điều khiển thông qua giao thức I2C. Việc sử dụng I2C giúp giảm số lượng chân GPIO cần thiết và đơn giản hóa sơ đồ kết nối phần cứng.

Các chân giao tiếp chính:

- SDA, SCL: kết nối với module I2C của vi điều khiển.
- Nguồn cấp: 5V.

LCD được khởi tạo trong hàm `main()` thông qua các hàm `lcd_init()`, `lcd_clear_display()` và các hàm điều khiển vị trí con trỏ.

1.3.3 Nguyên lý hiển thị

Việc cập nhật nội dung LCD được thực hiện trong hàm `updateLCD()`, được gọi liên tục trong vòng lặp chính của chương trình. Nội dung hiển thị trên LCD phụ thuộc vào trạng thái hiện tại của hệ thống (biến `status`) trong máy trạng thái hữu hạn (FSM).

LCD được chia làm 2 dòng với chức năng như sau:

- **Dòng 1:** Hiển thị chế độ hoạt động hiện tại của hệ thống (AUTO, MANUAL, SET TIME).

- **Dòng 2:** Hiển thị trạng thái đèn của hai tuyến giao thông hoặc thời gian cài đặt cho từng màu đèn.

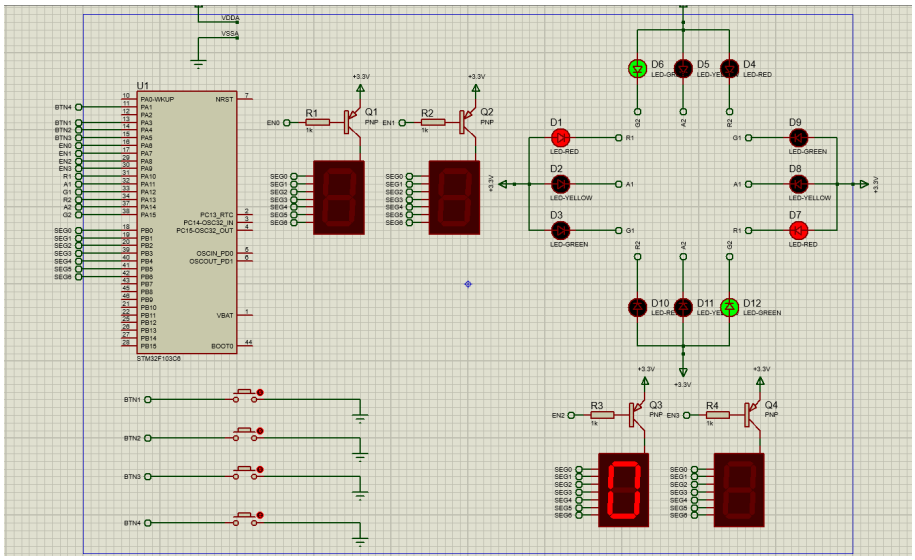
1.3.4 Nội dung hiển thị theo từng chế độ

- **Mode 1 – Automatic:** Hiển thị trạng thái đèn của hai tuyến (RED, GREEN, AMBER).
- **Mode 2 – Set RED time:** Hiển thị thời gian cài đặt cho đèn đỏ.
- **Mode 3 – Set AMBER time:** Hiển thị thời gian cài đặt cho đèn vàng.
- **Mode 4 – Set GREEN time:** Hiển thị thời gian cài đặt cho đèn xanh.
- **Manual Mode:** Hiển thị bước điều khiển hiện tại trong chế độ điều khiển bằng tay.

Nội dung LCD được cập nhật động theo sự thay đổi trạng thái và các thao tác nút nhấn của người dùng, đảm bảo thông tin hiển thị luôn đồng bộ với trạng thái thực tế của hệ thống.

2 Kiểm thử và mô phỏng

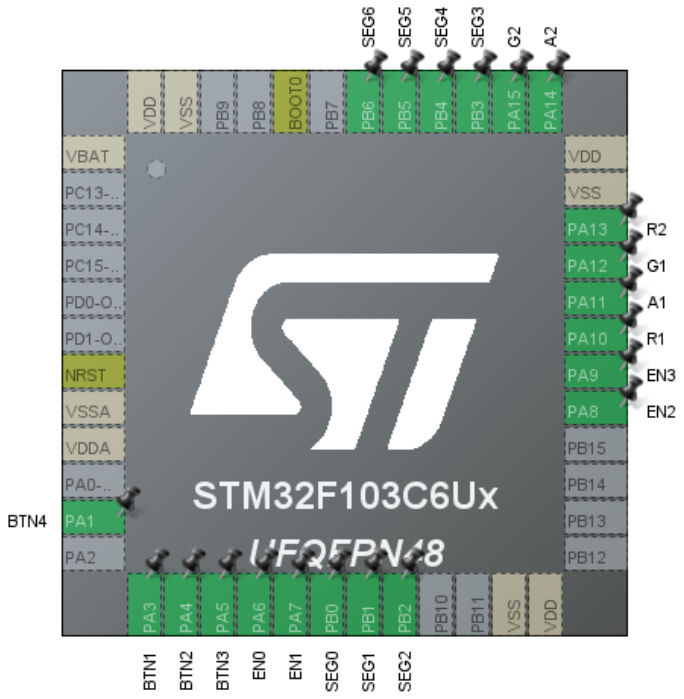
2.1 Dùng protues mô phỏng với STM32F103C6



Hình 6: Mô phỏng protues với STM32F103C6

2.2 Chạy trên mạch thật

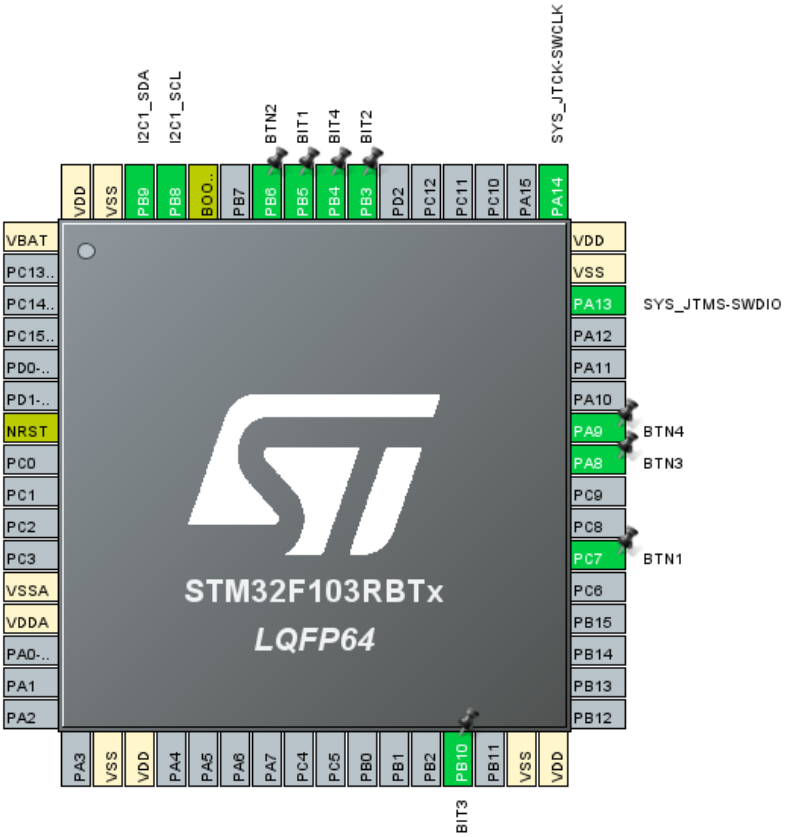
Vì lí do không đủ thiết bị để mô phỏng. Nhóm đã dùng STM32F103C6 và nối mạch với các các led và button rời rạc nên khi chuyển sang các module led và button nhóm đã sửa lại các hàm set để bật tắt đèn, còn cấu trúc còn lại không bị ảnh hưởng nhiều.



Hình 7: Nối chân với STM32F103C6

```
 void setRed1(){
 HAL_GPIO_WritePin(R1_GPIO_Port, R1_Pin, RESET);
 HAL_GPIO_WritePin(A1_GPIO_Port, A1_Pin, SET);
 HAL_GPIO_WritePin(G1_GPIO_Port, G1_Pin, SET);
}
void setGreen1(){
 HAL_GPIO_WritePin(R1_GPIO_Port, R1_Pin, SET);
 HAL_GPIO_WritePin(A1_GPIO_Port, A1_Pin, SET);
 HAL_GPIO_WritePin(G1_GPIO_Port, G1_Pin, RESET);
}
void setAmber1(){
 HAL_GPIO_WritePin(R1_GPIO_Port, R1_Pin, SET);
 HAL_GPIO_WritePin(A1_GPIO_Port, A1_Pin, RESET);
 HAL_GPIO_WritePin(G1_GPIO_Port, G1_Pin, SET);
}
void setRed2(){
 HAL_GPIO_WritePin(R2_GPIO_Port, R2_Pin, RESET);
 HAL_GPIO_WritePin(A2_GPIO_Port, A2_Pin, SET);
 HAL_GPIO_WritePin(G2_GPIO_Port, G2_Pin, SET);
}
void setGreen2(){
 HAL_GPIO_WritePin(R2_GPIO_Port, R2_Pin, SET);
 HAL_GPIO_WritePin(A2_GPIO_Port, A2_Pin, SET);
 HAL_GPIO_WritePin(G2_GPIO_Port, G2_Pin, RESET);
}
void setAmber2(){
 HAL_GPIO_WritePin(R2_GPIO_Port, R2_Pin, SET);
 HAL_GPIO_WritePin(A2_GPIO_Port, A2_Pin, RESET);
 HAL_GPIO_WritePin(G2_GPIO_Port, G2_Pin, SET);
}
```

Hình 8: Cách dùng set led trong khi mô phỏng với STM32F103C6



Hình 9: Nối chân với mạch thật

```
void setRed1() {
    HAL_GPIO_WritePin(BIT1_GPIO_Port, BIT1_Pin, SET);
    HAL_GPIO_WritePin(BIT2_GPIO_Port, BIT2_Pin, SET);
}
void setGreen1() {
    HAL_GPIO_WritePin(BIT1_GPIO_Port, BIT1_Pin, RESET);
    HAL_GPIO_WritePin(BIT2_GPIO_Port, BIT2_Pin, SET);
}
void setAmber1() {
    HAL_GPIO_WritePin(BIT1_GPIO_Port, BIT1_Pin, SET);
    HAL_GPIO_WritePin(BIT2_GPIO_Port, BIT2_Pin, RESET);
}
void setRed2() {
   HAL_GPIO_WritePin(BIT1_GPIO_Port, BIT1_Pin, RESET);
   HAL_GPIO_WritePin(BIT2_GPIO_Port, BIT2_Pin, RESET);
}
void setGreen2() {
   HAL_GPIO_WritePin(BIT1_GPIO_Port, BIT1_Pin, RESET);
   HAL_GPIO_WritePin(BIT2_GPIO_Port, BIT2_Pin, SET);
}
void setAmber2() {
   HAL_GPIO_WritePin(BIT1_GPIO_Port, BIT1_Pin, SET);
   HAL_GPIO_WritePin(BIT2_GPIO_Port, BIT2_Pin, RESET);
}
```

Hình 10: Cách dùng set led trong khi chạy mạch thật

3 Kết luận

Thông qua đồ án này, nhóm đã thiết kế và hiện thực thành công hệ thống điều khiển đèn giao thông dựa trên máy trạng thái hữu hạn (FSM) kết hợp với bộ định thời và cơ chế lập lịch tác vụ. Hệ thống đáp ứng đầy đủ các chức năng đã đề ra, bao gồm chế độ hoạt động bình thường, chế độ điều khiển bằng tay, chế độ cài đặt thời gian và chế độ ưu tiên cho người đi bộ.

Kết quả mô phỏng trên Proteus và thử nghiệm trên mạch phần cứng thực tế cho thấy hệ thống hoạt động ổn định, các trạng thái đèn chuyển đổi chính xác theo thiết kế và phản hồi tốt với các tín hiệu nút nhấn. Việc áp dụng mô hình FSM giúp chương trình có cấu trúc rõ ràng, dễ mở rộng và thuận tiện cho việc bảo trì trong các hệ thống nhúng có độ phức tạp cao.